



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Geoinformatyka**

Specjalność:

Michał Jan Wilkos

Nr albumu: 307370

Projekt i implementacja oprogramowania do analizy przestrzenno-czasowej danych treningowych z urządzeń GPS

Design and implementation of software for spatial-temporal
analysis of training data from GPS devices

Praca magisterska
napisana w Katedrze Oprogramowania Systemów Informatycznych
Instytutu Informatyki
pod kierunkiem dr Michała Klisowskiego

Lublin 2026

Spis treści

Wstęp	5
1 Teoria (do poprawy)	7
1.1 Pamięć (do poprawy)	7
1.1.1 Pamięć podręczna i hierarchia pamięci	7
1.1.2 Równoległe wykonywanie operacji	8
1.1.3 Wektoryzacja	9
1.2 GPS	11
1.2.1 Zasada działania	11
1.2.2 Formaty danych GPS	12
2 HPC i Python	17
2.1 HPC	17
2.1.1 Języki stosowane w HPC	19
2.1.2 Wbudowane struktury danych języka Python a wydajność obliczeniowa	19
Bibliografia	23

Wstęp

W czasach rosnącej świadomości na temat zdrowego trybu życia oraz dynamicznego rozwoju technologii pomiaru sesji treningowych coraz więcej osób decyduje się na monitorowanie swoich aktywności fizycznych, a popularność zegarków sportowych i urządzeń z modułami GPS rośnie z roku na rok. Jednak generowanie ogromnych ilości danych o przebytych dystansach i parametrach treningowych niesie ze sobą problemy nie tylko w kontekście interpretacji, ale przede wszystkim obliczeń, szczególnie w przypadku przetwarzania informacji przestrzenno-czasowych.

W procesie tworzenia oprogramowania analitycznego ważną rolę odgrywa wybór odpowiedniej metody przetwarzania dużych zbiorów danych. Chociaż język Python oferuje mnogość narzędzi, wiele rozwiązań różni się pod względem efektywności czasowej i pamięciowej. Często stosowane standardowe struktury danych, takie jak listy, mogą okazać się niewystarczające przy skomplikowanych i obciążających pamięć obliczeniach. Te zagadnienia oraz chęć sprawdzenia różnic w wydajności stanowiły inspirację do stworzenia aplikacji, która posłuży jako środowisko testowe dla tych metod.

Praca będzie się koncentrować na opracowaniu oraz zaimplementowaniu oprogramowania do analizy przestrzenno-czasowej danych treningowych, z uwzględnieniem porównania efektywności różnych metod algorytmów analitycznych. Głównym założeniem jest nie tylko stworzenie funkcjonalnego narzędzia, ale także zbadanie i zestawienie wydajności operacji wykonywanych na podstawowych strukturach danych w porównaniu do rozwiązań wektorowych.

Struktura pracy zostanie podzielona na trzy rozdziały. Pierwszy z nich omówi wykorzystaną technologię oraz charakterystykę przetwarzania danych GPS, ze wskazaniem na różnice pomiędzy klasycznym modelem programowania a podejściem wektorowym. Drugi rozdział będzie przedstawiać założenia projektowe aplikacji oraz metodologię przeprowadzania testów porównawczych. W ostatnim rozdziale przedstawione zostaną aspekty implementacyjne, w tym organizacja kodu oraz napotkane problemy, a jego ważną częścią będzie stanowić prezentacja wyników analizy porównawczej wydajności zastosowanych rozwiązań w odniesieniu do czasu przetwarzania i wykorzystania zasobów systemowych.

Rozdział 1

Teoria (do poprawy)

Ten rozdział poruszy tematy dotyczące pamięci, podstaw przetwarzania danych, które stanowią bazę do zrozumienia mechanizmów optymalizacji oprogramowania, a także przybliży architektury systemów nawigacji satelitarnej oraz różne formaty przechowujące dane przestrzenne.

1.1 Pamięć (do poprawy)

1.1.1 Pamięć podręczna i hierarchia pamięci

Współczesne procesory wykonują obliczenia szybciej, niż system pamięci jest w stanie dostarczyć im dane. Jest to problem znany jako „wąskie gardło” występujący w architekturze von Neumanna, według której dane są przechowywane w pamięci wraz z instrukcjami kodu programu[1]. Problem ten określa się również jako ścianę pamięci (ang. *memory wall*). To sprawia, że w aplikacjach przetwarzających duże ilości danych problemem staje się opóźnienie w dostępie do pamięci RAM, a nie moc obliczeniowa rdzenia.

Aby złagodzić ten problem, architektury komputerowe wykorzystują wielopoziomową pamięć podręczną, która stanowi swoisty bufor pomiędzy procesorem a pamięcią RAM. Dzieli się ona na trzy poziomy, z czego pierwszy z nich jest najszybszy, a trzeci – najwolniejszy[1]. Jest to mniejsza pamięć, ale znacznie szybsza od pamięci operacyjnej komputera. W celu uzyskania wysokiej wydajności często zarządza się danymi tak, aby procesor jak najczęściej znajdował potrzebne informacje w pamięci podręcznej, ograniczając kosztowne pobrania z pamięci głównej RAM.

Fundamentem wydajnego wykorzystania pamięci podręcznej jest zasada lokalności odwołań:

- Lokalność czasowa: Dane, do których uzyskano dostęp, prawdopodobnie będą potrzebne ponownie w niedługim czasie.

- Lokalność przestrzenna: Jeśli program odwołuje się do określonego adresu pamięci, z dużym prawdopodobieństwem wkrótce odwoła się do sąsiednich adresów[1].

Z perspektywy programisty i wyboru struktur danych w projekcie, lokalność przestrzenna ma bardzo ważne znaczenie. Procesory nie pobierają z pamięci pojedynczych bajtów, lecz całe linie pamięci podręcznej. Struktury takie jak tablice, gdzie elementy są ułożone w pamięci w kolejności jeden za drugim, są pod tym względem bardzo wydajne. Pobranie pierwszego elementu automatycznie ładuje do cache'u kolejne, co pozwala na płynne dostarczanie danych[2].

Struktury oparte na węzłach i wskaźnikach, takie jak listy połączone lub drzewa charakteryzują się rozrzuceniem elementów po sterzie pamięci. Przejście do kolejnego elementu wymaga skoku do losowego adresu, co często skutkuje chybieniem pamięci podręcznej i wstrzymaniem pracy procesora na czas pobrania danych z RAM[2].

Współczesne procesory wielordzeniowe zazwyczaj implementują trójpoziomowy system pamięci podręcznej. Pamięć podręczna pierwszego poziomu jest najszybszą i najmniejszą spośród wszystkich trzech poziomów. Dzieli się na dwie niezależne sekcje – L1i dla instrukcji oraz L1d dla danych. Taki podział zapobiega konfliktom między pobieraniem kodu a odczytem danych operacyjnych. Rozmiar tej pamięci waha się od 32 do 48 KB na rdzeń. [1]

Pamięć drugiego poziomu stanowi bufor między pamięcią L1 a wolniejszymi zasobami. Jej pojemność waha się od 256 KB do 2 MB na rdzeń. W przypadku „chybienia” w L1, procesor szuka danych w L2, co zapobiega wstrzymaniu pracy.[1]

Pamięć trzeciego poziomu jest największą pamięcią podręczną w procesorze. Zazwyczaj jest ona współdzielona przez wszystkie rdzenie w procesorze. Jej rozmiar może wynosić od kilkunastu do ponad stu megabajtów. Głównym zadaniem pamięci podręcznej trzeciego poziomu jest synchronizacja danych między rdzeniami, a także redukcja odwołań do pamięci RAM.[1]

Ważną kwestią do poruszenia w tym temacie jest mechanika transferu danych. System pamięci nie przesyła pojedynczych bajtów czy zmiennych. Komunikacja między pamięciami RAM, L3, L2 i L1 odbywa się za pomocą tzw. linii pamięci podręcznej (ang. *cache lines*). Oznacza to, że pojedyncze odwołanie do pamięci przez procesor powoduje pobranie całego 64-bajtowego bloku.[1]

1.1.2 Równoległe wykonywanie operacji

Przez dekady wzrost wydajności aplikacji opierał się na zwiększaniu częstotliwości taktowania procesorów, lecz skok technologiczny sprawił, że dalsze zwiększanie prędkości pojedynczego rdzenia stało się nieefektywne. Zamiast tego, zaczęto zwiększać liczbę rdzeni w procesorach, co doprowadziło do zmiany w projektowaniu systemów i aplikacji – przyspieszania pojedynczych instrukcji w stronę równoległych obliczeń[3].

Do klasyfikacji architektur równoległych stosuje się taksonomię zaproponowaną przez Michaela Flynna w 1966 roku

- SISD (ang. *Single Instruction, Single Data*): Architektura von Neumanna, gdzie jeden procesor wykonuje jedną instrukcję na jednej danej. Jest to model sekwencyjny.
- SIMD (ang. *Single Instruction, Multiple Data*): Jedna instrukcja sterująca wykonuje tę samą operację na wielu elementach danych jednocześnie. Jest to podstawa wektoryzacji.
- MISD (ang. *Multiple Instruction, Single Data*): Wiele instrukcji operuje na jednej danej.
- MIMD (ang. *Multiple Instruction, Multiple Data*): Wiele procesorów lub rdzeni wykonuje niezależnie różne instrukcje na różnych danych. Jest to podstawowy model równoległości[3].

Architektura MIMD jest realizowana w obecnych komputerach za pomocą procesorów wielordzeniowych. W kontekście analizy danych treningowych pozwala to na podział pracy na mniejsze, niezależne zadania. System może analizować kilka niezależnych plików jednocześnie, gdzie każdy plik obsługiwany jest przez osobny wątek procesora. W związku z tym system może działać w przykładowy sposób:

- Rdzeń 1 analizuje trasę biegu z poniedziałku.
- Rdzeń 2 analizuje trasę rowerową z wtorku.
- Rdzeń 3 analizuje spacer ze środy.

Ponieważ wynik analizy jednego działania nie zależy od wyniku drugiego, rdzenie te nie muszą się ze sobą komunikować ani na siebie czekać.[3]

1.1.3 Wektoryzacja

Według taksonomii zaproponowanej przez Michaela J. Flynna, tradycyjne jednostki obliczeniowe operują w modelu SISD. Oznacza to, że pojedyncza instrukcja maszynowa modyfikuje stan pojedynczego parametru w jednym cyklu maszynowym. Alternatywą, która zdominowała współczesne jednostki CPU, jest model SIMD. Paradygmat ten eksploatuje zjawisko równoległości na poziomie danych (ang. *Data-Level Parallelism*), umożliwiając zaaplikowanie tej samej operacji (np. dodawania, mnożenia) do wektora elementów rozlokowanych w dedykowanym, szerokim rejestrze sprzętowym.[3]

Zysk wydajnościowy płynący z wektoryzacji można opisać za pomocą prawa Amdahla. Jeżeli P oznacza ułamek programu podatny na zrównoleglenie wektorowe, a N

oznacza liczbę elementów przetwarzanych jednocześnie w jednym rejestrze wektorowym, to teoretyczne maksymalne przyspieszenie S (ang. *speedup*) określa wzór:

$$S = \frac{1}{(1 - P) + \frac{P}{N}}$$

Z powyższego równania wynika, że skuteczność technologii SIMD jest ściśle zdeterminowana przez to, jak duża część algorytmu daje się wyrazić w postaci niezależnych operacji wektorowych.[3]

Sama translacja instrukcji na postać wektorową nie gwarantuje proporcjonalnego przyspieszenia. Optymalizacja SIMD napotyka na dwa fundamentalne wyzwania, które muszą zostać uwzględnione w projektowaniu algorytmów. Najczęstszym powodem takiej nieefektywności jest niewystarczająca przepustowość szyny pamięci pomiędzy RAM a pamięcią podręczną procesora. Procesor jest w stanie wykonywać operacje SIMD znacznie szybciej, niż system pamięci dostarcza nowe dane.[3]

Bezpośrednią konsekwencją specyfiki ładowania danych wektorowych jest konieczność zmiany systemów projektowania struktur danych w pamięci (ang. *Data-Oriented Design*). Klasyczne programowanie obiektowe naturalnie układa dane w sposób AoS, czyli tablice struktur (ang. *Array of Structures*). Przylegające do siebie w pamięci bajty reprezentują różne atrybuty pojedynczego obiektu (np. x, y, z, kolor, czas). Utrudnia to załadowanie np. wyłącznie współrzędnych x z kolejnych elementów.

$$\text{AoS} = \left[\underbrace{(\text{Poz}_1, \text{Prę}_1)}_{\text{Obiekt}_1}, \underbrace{(\text{Poz}_2, \text{Prę}_2)}_{\text{Obiekt}_2}, \dots, \underbrace{(\text{Poz}_N, \text{Prę}_N)}_{\text{Obiekt}_N} \right] \quad (1.1)$$

Dlatego też aplikacje zorientowane na wysoką wydajność obliczeniową korzystają z ułożenia SoA, czyli struktury tablic (ang. *Structure of Arrays*), w której atrybuty tego samego typu tworzą oddzielne, ciągłe obszary pamięci.[4]

$$\text{SoA} = \begin{cases} \text{Tablica pozycji} = [\text{Poz}_1, \text{Poz}_2, \dots, \text{Poz}_N] \\ \text{Tablica prędkości} = [\text{Prę}_1, \text{Prę}_2, \dots, \text{Prę}_N] \end{cases} \quad (1.2)$$

Jeżeli algorytm odwołuje się do pamięci w sposób losowy lub nieliniowy, czyli tak jak ma to miejsce w przypadku układu tablic struktur, wczytana 64-bajtowa linia będzie zawierać wiele bezużytecznych w danym momencie informacji. To sprawia, że procesor szybko wykorzysta całą pojemność pamięci podręcznej pierwszego poziomu.

Z drugiej strony, gdy dane są ułożone w pamięci sekwencyjnie (układ struktur tablic), pobranie pierwszego elementu spowoduje załadowanie kolejnych elementów do pamięci L1. Dzięki temu rozwiązaniu procesor, wykonując instrukcje wektorowe, znajdzie kolejne potrzebne dane już w najszybszej pamięci.[4]

Aby zilustrować, w jaki sposób procesor i jego pamięć podręczna widzą układ tych danych podczas operacji wymagającej odczytu wyłącznie jednego atrybutu (np. samej

pozycji, z całkowitym pominięciem prędkości), można zestawić ze sobą oba podejścia. W przypadku tradycyjnego modelu AoS, wskaźnik przechodzący przez pamięć musi załadować pełen obiekt, odczytać pozycję, a następnie przeskoczyć niepotrzebną w danej chwili prędkość. Generuje to narzut i zanieczyszcza linię pamięci podręcznej:

$$\text{Odczyt AoS: } \left[\underbrace{\mathbf{Poz}_1}_{\text{odczyt}}, \underbrace{\text{Prę}_1}_{\text{omijane}}, \underbrace{\mathbf{Poz}_2}_{\text{odczyt}}, \underbrace{\text{Prę}_2}_{\text{omijane}}, \dots, \underbrace{\mathbf{Poz}_N}_{\text{odczyt}}, \underbrace{\text{Prę}_N}_{\text{omijane}} \right] \quad (1.3)$$

Z kolei w architekturze SoA odczytywane są wyłącznie pożądane informacje. Procesor sprawnie pobiera do rejestrów spójny, ciągły wektor jednorodnych danych, podczas gdy niezwiązane z daną operacją tablice pozostają całkowicie nietknięte i nie obciążają przepustowości pamięci:

$$\text{Odczyt SoA: } \left\{ \begin{array}{l} \left[\mathbf{Poz}_1, \mathbf{Poz}_2, \dots, \mathbf{Poz}_N \right] \\ \text{ciągły odczyt (tylko potrzebne dane)} \\ \left[\text{Prę}_1, \text{Prę}_2, \dots, \text{Prę}_N \right] \\ \text{całkowicie nietknięte bloki pamięci} \end{array} \right. \quad (1.4)$$

1.2 GPS

Global Positioning System (GPS), oficjalnie znany jako NAVSTAR GPS, to satelitalny system nawigacyjny stworzony i zarządzany przez Departament Obrony Stanów Zjednoczonych. Należy on do szerszej rodziny systemów klasy GNSS (ang. *Global Navigation Satellite System*), do której zaliczają się również europejski system Galileo, rosyjski GLONASS oraz chiński BeiDou. Choć w mowie potocznej akronim „GPS” stosuje się uniwersalnie do określenia każdego rodzaju nawigacji satelitarnej, w dosłownym ujęciu odnosi się on do konkretnego amerykańskiego systemu satelitarnego.[5]

1.2.1 Zasada działania

Zasada działania opiera się na pomiarze czasu propagacji sygnału. Jeżeli odbiornik wie, o której godzinie sygnał został wysłany przez satelitę i o której dotarł do anteny, może obliczyć czas przelotu (ang. *Time of Flight*). Odległość od satelity to iloczyn czasu przelotu i prędkości światła. Odległość ta determinuje miejsce w przestrzeni, gdzie znajduje się odbiornik.[5]

W ujęciu matematycznym, jeśli satelita emituje sygnał w czasie t^s , a odbiornik rejestruje go w czasie t^r , to teoretyczna odległość geometryczna d jest wyrażana wzorem:

$$d = c \cdot (t^r - t^s) \quad (1.5)$$

gdzie c oznacza prędkość światła. W praktyce jednak metoda ta wymaga uwzględnienia dodatkowych czynników. Wyznaczenie odległości od jednego satelity lokalizuje odbiornik na powierzchni sfery. Zastosowanie dwóch satelitów zawęża pozycję do okręgu stanowiącego przecięcie dwóch sfer, a z trzech – do dwóch punktów w przestrzeni.

Aby jednoznacznie określić swoją pozycję w trójwymiarowym układzie współrzędnych (np. WGS 84), odbiornik musi zatem wyznaczyć cztery niewiadome: współrzędne przestrzenne x, y, z oraz błąd zegara δt . Wymaga to jednoczesnego odbioru sygnału z minimum czterech satelitów, co pozwala na ułożenie i rozwiązanie układu równań składającego się z czterech równań nieliniowych w postaci:[5]

$$\rho = \sqrt{(x_1 - x)^2 + (y_1 - y)^2 + (z_1 - z)^2} + c \cdot \delta t \quad (1.6)$$

1.2.2 Formaty danych GPS

Wraz z popularyzacją systemów informacji geograficznej (GIS) i osobistych urządzeń nawigacyjnych, zaistniała potrzeba ustandaryzowania zapisu w sposób umożliwiający łatwą wymianę informacji między różnymi platformami. Doprowadziło to do powstania formatów opartych na języku znaczników XML (ang. *Extensible Markup Language*), z których najpowszechniejszym stał się GPX (ang. *GPS Exchange Format*).

GPX zdefiniował podstawowy system opisu tras (ang. *Routes*), śladów (ang. *Tracks*) i punktów (*Waypoints*). Jednakże, wraz z rozwojem telemetrii sportowej i smartwatchy, nawigacyjny charakter formatu GPX okazał się niewystarczający, co wymusiło stworzenie nowych formatów, integrujących dane przestrzenne z fizjologicznymi.[6]

Ewolucja ta poszła w dwóch głównych kierunkach. Z jednej strony, na potrzeby urządzeń sportowych opracowano wydajny, binarny format FIT (ang. *Flexible and Interoperable Data Transfer*). Dzięki swojej skompresowanej strukturze pozwala on na bardzo oszczędny zapis danych lokalizacyjnych oraz parametrów biomechanicznych i fizjologicznych (takich jak moc czy tętno) bezpośrednio w pamięci zegarków i liczników rowerowych.[7] Z drugiej strony, naturalnym krokiem było rozwinięcie dotychczasowej technologii XML w stronę sportu. Tak powstał format TCX (ang. *Training Center XML*), który stał się bezpośrednim łącznikiem między czysto geograficznym GPX-em a nowoczesną telemetrią treningową.

Format TCX (Training Center XML) został zaprojektowany i wprowadzony przez firmę Garmin w 2007 roku jako część oprogramowania Garmin Training Center. W przeciwieństwie do uniwersalnego formatu GPX, TCX został zbudowany od podstaw z myślą o analizie sportowej, rekreacji i monitorowaniu wysiłku fizycznego.

Z punktu widzenia inżynierii danych, plik TCX jest dokumentem XML podlegającym walidacji przez schemat XSD (XML Schema Definition), opublikowany w przestrzeni nazw <http://www.garmin.com/xmlschemas/TrainingCenterDatabase/v2>. [8]

Listing 1.1: Przykładowy plik TCX

```

1 <?xml version="1.0" encoding="UTF-8"?>
2 <TrainingCenterDatabase
3   xsi:schemaLocation="http://www.garmin.com/xmlschemas/
   TrainingCenterDatabase/v2 http://www.garmin.com/xmlschemas/
   TrainingCenterDatabasev2.xsd"
4   xmlns:ns5="http://www.garmin.com/xmlschemas/ActivityGoals/v1"
5   xmlns:ns3="http://www.garmin.com/xmlschemas/ActivityExtension/
   v2"
6   xmlns:ns2="http://www.garmin.com/xmlschemas/UserProfile/v2"
7   xmlns="http://www.garmin.com/xmlschemas/TrainingCenterDatabase
   /v2"
8   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
   xmlns:ns4="http://www.garmin.com/xmlschemas/
   ProfileExtension/v1">
9 <Activities>
10 <Activity Sport="Running">
11 <Id>2025-05-06T04:58:19.000Z</Id>
12 <Lap StartTime="2025-05-06T04:58:19.000Z">
13 <TotalTimeSeconds>335.479</TotalTimeSeconds>
14 <DistanceMeters>1000.0</DistanceMeters>
15 <MaximumSpeed>3.4149999618530273</MaximumSpeed>
16 <Calories>58</Calories>
17 <AverageHeartRateBpm>
18 <Value>146</Value>
19 </AverageHeartRateBpm>
20 <MaximumHeartRateBpm>
21 <Value>158</Value>
22 </MaximumHeartRateBpm>
23 <Intensity>Active</Intensity>
24 <TriggerMethod>Manual</TriggerMethod>
25 <Track>
26 <Trackpoint>
27 <Time>2025-05-06T04:58:19.000Z</Time>
28 <Position>
29 <LatitudeDegrees>51.23510423116386</
   LatitudeDegrees>
30 <LongitudeDegrees>22.55628313869238</

```

```

31         LongitudeDegrees>
32     </Position>
33     <AltitudeMeters>171.39999389648438</AltitudeMeters>
34     <DistanceMeters>0.8999999761581421</DistanceMeters>
35     <HeartRateBpm>
36         <Value>113</Value>
37     </HeartRateBpm>
38     <Extensions>
39         <ns3:TPX>
40             <ns3:Speed>0.8679999709129333</ns3:Speed>
41             <ns3:RunCadence>0</ns3:RunCadence>
42         </ns3:TPX>
43     </Extensions>
44     </Trackpoint>
45     <Trackpoint>
46     </Track>
47 </Lap>
48 </Activity>
49 </Activities>
50 </TrainingCenterDatabase>

```

Główną cechą odróżniającą TCX od innych formatów jest jego orientacja na „aktywność” (ang. *Activity*), a nie tylko i wyłącznie na „ścieżkę” (ang. *Track*). Plik TCX modeluje zbiór danych telemetrycznych w oparciu o zagnieżdżoną hierarchię:

- TrainingCenterDatabase: Główny węzeł dokumentu, który zawiera w sobie aktywności użytkownika.
- Activities / Activity: Reprezentuje pojedynczy trening. Atrybut Sport węzła Activity natychmiast kategoryzuje rodzaj ruchu (np. Running, Biking).
- Lap (Okrążenie): Jednostka podziału treningu w TCX. TCX dzieli aktywność na logiczne okrążenia (definiowane ręcznie przez użytkownika za pomocą przycisku na zegarku lub automatycznie, np. co 1 kilometr). Węzeł Lap przechowuje od razu zagregowane statystyki: TotalTimeSeconds (całkowity czas), DistanceMeters (dystans), MaximumSpeed (prędkość maksymalna) oraz Calories (spalone kalorie) itp.
- Track / Trackpoint: Wewnątrz każdego okrążenia znajduje się ścieżka, stanowiąca zbiór punktów pomiarowych (Trackpoint).[8]

Pojedynczy punkt pomiarowy w formacie TCX stanowi połączenie danych z różnych sensorów. Podczas gdy tradycyjny odbiornik GPS dostarcza głównie współrzędnych i

czasu, zegarek sportowy synchronizuje w tym samym momencie dane z pulsometru, barometru i akcelerometru.

Struktura informacyjna węzła Trackpoint obejmuje:

- `<Time>`: Znacznik czasu w formacie ISO 8601 (np. 2023-05-14T08:30:00.000Z).
- `<Position>`: Blok przestrzenny zawierający `<LatitudeDegrees>` i `<LongitudeDegrees>`. Reprezentacja bazuje na układzie WGS 84 w formie stopni dziesiętnych. Element ten jest opcjonalny – np. w przypadku biegu na bieżni mechanicznej, zegarek generuje punkty TCX pozbawione bloku `Position`, ale zawierające tętno i czas.
- `<AltitudeMeters>`: Wysokość, najczęściej z dokładnego czujnika barometrycznego kalibrowanego sygnałem GPS.
- `<DistanceMeters>`: Skumulowany dystans od początku okrążenia.
- `<HeartRateBpm>`: Informacja o tętnie w uderzeniach na minutę.[8]

Z uwagi na fakt, że oryginalny schemat TCX powstał kilkanaście lat temu, nie przewidywał on nowoczesnych metryk, takich jak moc generowana podczas biegu, balans stóp czy saturacja krwi. Aby uniknąć konieczności ciągłego modyfikowania głównego schematu, Garmin wprowadził mechanizm rozszerzeń w oparciu o nowe przestrzenie nazw (`ActivityExtension/v2`). Wewnątrz węzła Trackpoint można umieścić blok `<Extensions>`, który pozwala na dołączanie dowolnych nowych danych.[8]

Rozdział 2

HPC i Python

Ten rozdział poruszy tematy dotyczące architektury systemów High-Performance Computing (HPC) i specyfiki języka Python w celu zrozumienia jego ograniczeń wydajnościowych. Przybliżone zostaną również metody integracji standardowej implementacji z wydajnymi bibliotekami opartymi na niskopoziomowych rozwiązaniach.

2.1 HPC

Przetwarzanie o wysokiej wydajności (ang. *High-Performance Computing*) to dziedzina informatyki zajmująca się agregacją mocy obliczeniowej w celu rozwiązywania złożonych problemów naukowych i inżynierskich, których realizacja na zwykłych komputerach byłaby niemożliwa ze względu na ograniczenia czasowe lub pamięciowe.

Systemy HPC, potocznie nazywane superkomputerami, pozwalają na wykonywanie bilionów, a nawet trylionów obliczeń w ciągu sekundy. Podstawową miarą wydajności w systemach HPC jest wskaźnik FLOPS (ang. *Floating-Point Operations Per Second*), określający liczbę operacji zmiennoprzecinkowych wykonywanych w ciągu jednej sekundy. Współczesne superkomputery osiągają wydajność rzędu petaFLOPS (10^{15} operacji na sekundę), a najpotężniejsze jednostki na świecie przekroczyły już barierę exaFLOPS (10^{18} operacji na sekundę).[9]

Współczesne systemy HPC rzadko są pojedynczymi, monolitycznymi maszynami. Opierają się na architekturze klastrowej, w której setki lub tysiące niezależnych serwerów jest połączonych w jedną, zintegrowaną strukturę. Podstawowe elementy sprzętowe klastra HPC obejmują:

- Węzły obliczeniowe (ang. *Compute Nodes*): Odpowiedzialne za wykonywanie właściwych obliczeń. Każdy węzeł to w praktyce autonomiczny serwer wyposażony we własne procesory wielordzeniowe, pamięć operacyjną oraz coraz częściej w akcelerator obliczeniowe, takie jak procesory graficzne.[10]

- Węzły dostępne (ang. *Login Nodes*): Są to punkty wejściowe do systemu. Użytkownik loguje się do węzła dostępowego, aby kompilować kod, zarządzać swoimi danymi oraz zlecać zadania obliczeniowe. Węzły te nie służą do wykonywania ciężkich obliczeń, aby nie blokować dostępu innym użytkownikom.[10]
- Sieć połączeń: Ponieważ zadania w HPC są dzielone na wiele węzłów, które muszą nieustannie wymieniać między sobą dane, standardowe sieci Ethernet mogą okazać się niewystarczające. W systemach HPC stosuje się dedykowane sieci charakteryzujące się bardzo wysoką przepustowością i niskimi opóźnieniami. Dominującą technologią w tym obszarze jest InfiniBand.[3]
- Systemy pamięci masowej: Obliczenia HPC generują i konsumują terabajty danych. Zwykle dyski twarde nie są w stanie poradzić sobie z takim wyzwaniem. Stosuje się więc systemy plików (np. Lustre), które pozwalają tysiącom procesów na jednoczesny odczyt i zapis danych, rozpraszając fizyczny zapis na setki dysków i serwerów plikowych.[10]

Aby w pełni wykorzystać potencjał klastra HPC, oprogramowanie musi zostać zaprojektowane tak, by działało równolegle. Wyróżnia się trzy główne paradygmaty programowania w takich środowiskach:

- Pamięć współdzielona (ang. *Shared Memory*): Wykorzystywana w obrębie pojedynczego węzła. Wiele wątków tego samego programu wykonuje się na różnych rdzeniach procesora, ale wszystkie mają dostęp do tej samej fizycznej przestrzeni pamięci. Standardem programistycznym dla tego modelu jest OpenMP.[3]
- Pamięć rozproszona (ang. *Distributed Memory*): Konieczna, gdy program wykonuje się na wielu różnych węzłach, z których każdy posiada własną, niezależną pamięć. Węzły muszą „rozmawiać” ze sobą, przesyłając komunikaty przez sieć połączeń. De facto standardem w tej dziedzinie jest biblioteka MPI (ang. *Message Passing Interface*).[3]
- Przetwarzanie heterogeniczne (ang. *Heterogeneous Computing*): Gdy obliczenia zlecano są do kart graficznych, stosuje się specyficzne języki i biblioteki, takie jak CUDA (dla środowiska NVIDIA) lub otwarty standard OpenCL.[3]

W środowisku HPC kod źródłowy to jedynie połowa sukcesu. Równie istotny jest sam proces tłumaczenia go na język maszynowy. Ośrodki superkomputerowe rzadko opierają się wyłącznie na standardowych kompilatorach typu open-source, często inwestując w kompilatory dedykowane konkretnym architekturze sprzętowej.

Kompilatory stworzone dla takich firm jak Intel, NVIDIA, czy AMD są w stanie wygenerować zestaw instrukcji dopasowany do mikroarchitektury procesora, na którym kod będzie uruchamiany.

2.1.1 Języki stosowane w HPC

W systemach HPC priorytetem jest absolutna maksymalizacja wydajności oraz minimalizacja czasu wykonania i zużycia pamięci. Z tego powodu dominującą rolę odgrywają języki kompilowane (tzw. języki niskiego poziomu), które oferują programiście bezpośrednią kontrolę nad zarządzaniem pamięcią operacyjną oraz architekturą sprzętową, a jednocześnie pozwalają kompilatorom na przeprowadzanie znacznej optymalizacji kodu wynikowego.

Język C jest powszechnie stosowany ze względu na wysoką wydajność, zwężłość składni i minimalny narzut abstrakcji (ang. *overhead*). Operowanie na poziomie wskaźników pozwala na ścisłą integrację kodu z architekturą sprzętową. Ułatwia to optymalizację algorytmów oraz pozwala kompilatorom na skuteczną wektoryzację pętli. Ponadto język C pełni rolę uniwersalnego standardu w tworzeniu interfejsów binarnych, co umożliwia bezproblemową integrację oprogramowania i bibliotek tworzonych w innych technologiach.

Z kolei język C++ rozwija te możliwości, łącząc bazową wydajność języka C z paradygmatami programowania obiektowego. W kontekście HPC istotne znaczenie ma zastosowanie szablonów, które pozwalają na przeniesienie części logiki decyzyjnej z czasu wykonania programu na czas jego kompilacji. Umożliwia to projektowanie elastycznych struktur danych bez dodatkowego obciążenia wydajnościowego. Współczesne standardy języka C++ dostarczają również natywne mechanizmy współbieżności, wspierając tym samym rozwój zaawansowanych aplikacji wielowątkowych.[11]

2.1.2 Wbudowane struktury danych języka Python a wydajność obliczeniowa

Język Python, dzięki swojej elastyczności, dynamicznemu typowaniu oraz niezwykle czytelnej składni, stał się jednym z najpopularniejszych języków programowania na świecie. Jednak mechanizmy, które czynią go tak przystępnym w procesie tworzenia oprogramowania, stanowią jednocześnie jego największą barierę w kontekście przetwarzania o wysokiej wydajności. Standardowe implementacje opierają się na architekturze, która przedkłada uniwersalność nad optymalizację sprzętową.

W językach niskiego poziomu, takich jak C czy C++, standardowa tablica jest ciągłym blokiem pamięci przechowującym wartości jednego, z góry określonego typu (np. 64-bitowe liczby całkowite), co zapewnia lokalność przestrzenną oraz odgórnie znany rozmiar. Wbudowana lista w języku Python działa na zupełnie innej zasadzie. Pod względem implementacyjnym nie jest to tablica przechowująca bezpośrednio dane, lecz dynamiczna tablica wskaźników. Oznacza to, że każdy element na liście jest jedynie adresem lub inaczej odniesieniem do rzeczywistego obiektu rozsianego w pamięci operacyjnej komputera.[12]

Dodatkowo, z uwagi na dynamiczne typowanie i fakt, że w Pythonie wszystko jest

obiektem, nawet prosta liczba całkowita nie jest tylko ciągiem bitów. W standardowej implementacji Pythona, każda liczba jest strukturą *PyObject*, która oprócz samej wartości zawiera minimum dwa dodatkowe pola:

- Licznik referencji (ang. *reference count*) – przechowujący informację o liczbie zmiennych wskazujących na dany obiekt.
- Wskaźnik na typ obiektu – pozwalający interpreterowi określić w czasie wykonywania, z jakim typem danych ma do czynienia.[12]

Taka architektura generuje spory narzut pamięciowy (ang. *memory overhead*). Podczas gdy pojedyncza, 64-bitowa liczba całkowita w języku C zajmuje dokładnie 8 bajtów, ten sam obiekt w Pythonie może zajmować znacznie więcej bajtów w zależności od potrzeby, a do tego dochodzą jeszcze 8-bajtowe wskaźniki w samej liście. W przypadku przetwarzania milionów punktów z danych GPS, ilość alokowanej pamięci drastycznie rośnie, obciążając przepustowość szyny pamięci.

Jeszcze poważniejszą konsekwencją jest wpływ list na pamięć podręczną procesora. Jak wspomniano w rozdziale poświęconym hierarchii pamięci, procesory osiągają maksymalną wydajność, gdy dane są ułożone sekwencyjnie. Przetwarzanie natywnej listy w Pythonie zmusza procesor do wykonywania skoków. Odczyt elementu wymaga najpierw pobrania wskaźnika, a następnie wyciągnięcie informacji o nim, co prowadzi do odczytu danych z losowego, nieprzewidywalnego adresu w pamięci RAM.[4]

Prowadzi to do nieustannych ”chybień” pamięci podręcznej (ang. *cache misses*). Linie pamięci cache są wypełniane bezużytecznymi danymi, a procesor spędza większość cykli zegara na bezczynnym oczekiwaniu na dostarczenie danych z wolnej pamięci głównej RAM. Architektura ta jest klasycznym przykładem skrajnie rozproszonego układu, przeciwnego do optymalnego wzorca Structure of Arrays (SoA).[2]

Z perspektywy wektoryzowania lub zrównoleglenia na poziomie danych, natywne listy Pythona są praktycznie nieużyteczne dla kompilatorów i procesorów wektorowych. Mechanizmy wektoryzacji wymagają, aby instrukcje były aplikowane do paczek jednorodnych, przylegających do siebie w pamięci danych [3].

Ponieważ lista w Pythonie może przechowywać elementy różnych typów w sposób dynamiczny (np. liczby całkowite, zmiennoprzecinkowe i ciągi znaków obok siebie), kompilator JIT lub sprzęt nie są w stanie z góry zagwarantować spójności typów w czasie wykonania. Przed każdą operacją arytmetyczną interpreter musi przeprowadzić kosztowną procedurę dynamicznego sprawdzania typów (ang. *type checking*) dla każdego pojedynczego obiektu. To skutecznie blokuje możliwość wykorzystania szerokich rejestrów wektorowych współczesnych architektur HPC.[13]

W celu ominięcia tych barier architektonicznych, biblioteki analityczne zorientowane na wydajność zmuszone są do omijania wewnętrznych struktur Pythona i implem-

wania własnych, ciągłych bloków pamięci w niższych warstwach (np. poprzez język C), co stanowi fundament działania takich narzędzi jak NumPy.

Bibliografia

- [1] Ulrich Drepper, *What Every Programmer Should Know About Memory*, Red Hat, Inc., 2007
- [2] Agner Fog, *Optimizing software in C++: An optimization guide for Windows, Linux, and Mac platforms*, Agner Fog, 2025
- [3] John L. Hennessy, J. L., David A. Patterson, *Computer Architecture: A Quantitative Approach Fifth Edition*, Morgan Kaufmann, 2017
- [4] Richard Acton, *Data-Oriented Design: Software engineering for limited resources and short schedules*, Richard Acton, 2018
- [5] Bernhard Hofmann-Wellenhof, Herbert Lichtenegger, Elmar Wasle, *GNSS – Global Navigation Satellite Systems: GPS, GLONASS, Galileo, and more*, SpringerWien-NewYork, Austria, 2008
- [6] TopoGrafix, *GPX: the GPS Exchange Format*, <https://www.topografix.com/gpx.asp>, [dostęp: 14.04.2026]
- [7] Garmin, *Flexible and Interoperable Data Transfer (FIT) SDK*, <https://developer.garmin.com/fit/overview/>
- [8] Garmin, *Training Center Database XML Schema*, <https://www8.garmin.com/xmlschemas/TrainingCenterDatabasev2.xsd>, [dostęp: 14.04.2026]
- [9] TOP500, *TOP500's The List*, <https://www.top500.org/lists/top500/2025/11/>, [dostęp: 06.05.2026]
- [10] ETH Zurich, *HPC Documentation*, <https://docs.hpc.ethz.ch/>, [dostęp: 06.05.2026]
- [11] Björn Andrist, Viktor Sehr, *C++ High Performance: Master the art of optimizing the performance of your C++ applications*, Packt Publishing, Birmingham, 2018
- [12] Python Software Foundation, *Python 3 Documentation*, <https://docs.python.org/3/>, [dostęp: 06.05.2026]

- [13] Kevin Williams, Jason McCandless, David Gregg, *Dynamic Interpretation for Dynamic Scripting Languages*, Department of Computer Science, Trinity College Dublin, Dublin, 2009